AI Assisted Coding
Assignment - 11.1
2303A510A0
Batch - 14

Task - 1 : Stack Implementation

Prompt :

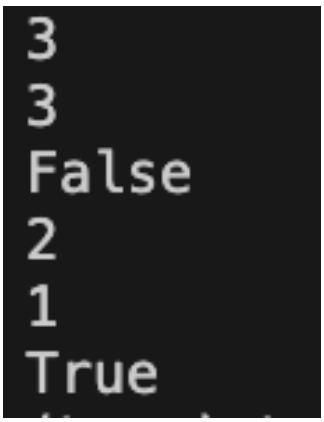
generate a Stack class with push, pop, peek, and is_empty methods.
give all required methods and Doc strings for the Stack class.

Code :

Ai-11.1.py > Stack > pop

```python
# generate a Stack class with push, pop, peek, and is_empty methods.
# give all required methods and Doc strings for the Stack class.
class Stack:
    """
    A Stack is a data structure that follows the Last In First Out (LIFO) principle.
    It supports operations to add, remove, and inspect elements in the stack.
    """
    def __init__(self):
        """Initialize an empty stack."""
        self.stack = []
    def push(self, item):
        """
        Add an item to the top of the stack.
        Parameters:
        item: The item to be added to the stack.
        """
        self.stack.append(item)
    def pop(self):
        """
        Remove and return the item at the top of the stack.
        Returns:
        The item at the top of the stack.
        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Pop from an empty stack")
        return self.stack.pop()
    def peek(self):
        """
        Return the item at the top of the stack without removing it.
        Returns:
        The item at the top of the stack.
        Raises:
        IndexError: If the stack is empty.
        """
        if self.is_empty():
            raise IndexError("Peek from an empty stack")
        return self.stack[-1]
    def is_empty(self):
        """
        Check if the stack is empty.
        Returns:
        True if the stack is empty, False otherwise.
        """
        return len(self.stack) == 0
# Example usage:
if __name__ == "__main__":
    stack = Stack()
    stack.push(1)
    stack.push(2)
    stack.push(3)
    print(stack.peek())  # Output: 3
    print(stack.pop())   # Output: 3
    print(stack.is_empty())  # Output: False
    print(stack.pop())   # Output: 2
    print(stack.pop())   # Output: 1
    print(stack.is_empty())  # Output: True
```

OutPut :



Task 2 : Queue Implementation

Prompt :

implement a Queue using Python lists
FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Code:

Ai-11.1.py > ...

```python
# implement a Queue using Python lists
# FIFO-based queue class with enqueue, dequeue, peek, and size methods.
class Queue:
    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the item at the front of the queue."""
        if not self.is_empty():
            return self.items.pop(0)
        else:
            raise IndexError("Dequeue from an empty queue")

    def peek(self):
        """Return the item at the front of the queue without removing it."""
        if not self.is_empty():
            return self.items[0]
        else:
            raise IndexError("Peek from an empty queue")

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)

    def is_empty(self):
        """Check if the queue is empty."""
        return len(self.items) == 0
# Example usage:
if __name__ == "__main__":
    q = Queue()
    q.enqueue(1)
    q.enqueue(2)
    q.enqueue(3)

    print("Front item:", q.peek())  # Output: Front item: 1
    print("Queue size:", q.size())   # Output: Queue size: 3

    print("Dequeue item:", q.dequeue())  # Output: Dequeue item: 1
    print("Front item after dequeue:", q.peek())  # Output: Front item after dequeue: 2
    print("Queue size after dequeue:", q.size())   # Output: Queue size after dequeue: 2
```

OutPut:

```
Front item: 1
Queue size: 3
Dequeue item: 1
Front item after dequeue: 2
Queue size after dequeue: 2
```

Task 3 : Linked List

Prompt :

Generate a Singly Linked List with insert and display methods.
'''Sample Input Code:
class Node:
pass
class LinkedList:
pass'''
Give linked list implementation with clear method documentation.

Ai-11.1.py > ...

```python
# generate a Singly Linked List with insert and display methods.
'''Sample Input Code:
class Node:
pass
class LinkedList:
pass'''
# Give linked list implementation with clear method documentation.
class Node:
    """A class representing a node in a singly linked list."""
    def __init__(self, data):
        """Initialize the node with data and set the next pointer to None."""
        self.data = data
        self.next = None
class LinkedList:
    """A class representing a singly linked list."""
    def __init__(self):
        """Initialize the linked list with head set to None."""
        self.head = None

    def insert(self, data):
        """Insert a new node with the given data at the end of the linked list."""
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        last_node = self.head
        while last_node.next:
            last_node = last_node.next
        last_node.next = new_node

    def display(self):
        """Display the contents of the linked list."""
        current_node = self.head
        while current_node:
            print(current_node.data, end=' ')
            current_node = current_node.next
        print()  # for a new line after displaying the list
# Example usage:
if __name__ == "__main__":
    linked_list = LinkedList()
    linked_list.insert(10)
    linked_list.insert(20)
    linked_list.insert(30)
    print("Linked List contents:")
    linked_list.display()
```

OutPut:

```
Linked List contents:
10 20 30
```

Task - 4 : Binary Search Tree (BST)

Prompt :

generate Binary Search Tree (BST) with insert and in-order traversal methods.
"""

Sample Input Code:
class BST:
pass
"""

BST implementation with recursive insert and traversal methods.

Code:

Ai-11.1.py > BST > _insert_recursive

```python
# generate Binary Search Tree (BST) with insert and in-order traversal methods.
"""
Sample Input Code:
class BST:
pass
"""
# BST implementation with recursive insert and traversal methods.
class BSTNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None
    def insert(self, value):
        if self.root is None:
            self.root = BSTNode(value)
        else:
            self._insert_recursive(self.root, value)
    def _insert_recursive(self, node, value):
        if value < node.value:
            if node.left is None:
                node.left = BSTNode(value)
            else:
                self._insert_recursive(node.left, value)
        else:
            if node.right is None:
                node.right = BSTNode(value)
            else:
                self._insert_recursive(node.right, value)
    def in_order_traversal(self):
        return self._in_order_recursive(self.root)
    def _in_order_recursive(self, node):
        result = []
        if node is not None:
            result.extend(self._in_order_recursive(node.left))
            result.append(node.value)
            result.extend(self._in_order_recursive(node.right))
        return result
# Example usage:
if __name__ == "__main__":
    bst = BST()
    bst.insert(5)
    bst.insert(3)
    bst.insert(7)
    bst.insert(2)
    bst.insert(4)
    bst.insert(6)
    bst.insert(8)
    print("In-order Traversal:", bst.in_order_traversal())
```

OutPut:

```
In-order Traversal: [2, 3, 4, 5, 6, 7, 8]
```

Task - 5 : Hash Table

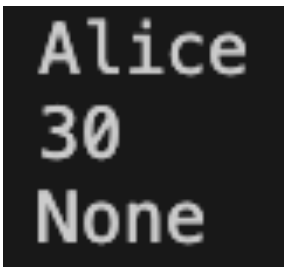
Prompt :

generate hash table with basic insert, search, and delete methods.
'''Sample Input Code:
class HashTable:
pass'''
Collision handling using chaining, with well-commented methods.

Ai-11.1.py > HashTable > __init__

```python
# generate  hash table with basic insert, search, and delete methods.
'''Sample Input Code:
class HashTable:
pass'''
# Collision handling using chaining, with well-commented methods.
class HashTable:
    def __init__(self):
        # Initialize the hash table with a fixed size and empty buckets
        self.size = 10
        self.table = [[] for _ in range(self.size)]
    def hash_function(self, key):
        # Simple hash function to compute the index for a given key
        return hash(key) % self.size
    def insert(self, key, value):
        # Insert a key-value pair into the hash table
        index = self.hash_function(key)
        bucket = self.table[index]
        # Check if the key already exists in the bucket
        for i, (k, v) in enumerate(bucket):
            if k == key:
                # If key exists, update the value
                bucket[i] = (key, value)
                return
        # If key does not exist, add a new key-value pair to the bucket
        bucket.append((key, value))
    def search(self, key):
        # Search for a value by its key in the hash table
        index = self.hash_function(key)
        bucket = self.table[index]
        # Iterate through the bucket to find the key
        for k, v in bucket:
            if k == key:
                return v  # Return the value if key is found
        return None  # Return None if key is not found
    def delete(self, key):
        # Delete a key-value pair from the hash table
        index = self.hash_function(key)
        bucket = self.table[index]
        # Iterate through the bucket to find and remove the key-value pair
        for i, (k, v) in enumerate(bucket):
            if k == key:
                del bucket[i]  # Remove the key-value pair from the bucket
                return True  # Return True if deletion is successful
        return False  # Return False if key is not found
# Example usage:
hash_table = HashTable()
hash_table.insert("name", "Alice")
hash_table.insert("age", 30)
print(hash_table.search("name"))  # Output: Alice
print(hash_table.search("age"))   # Output: 30
hash_table.delete("name")
print(hash_table.search("name"))  # Output: None
```

OutPut :



Task - 6 : Graph Representation

Prompt :

generate the code to impliment a graph using an adjacency list.
Graph with methods to add vertices, add edges, and display connections.

Code:

Ai-11.1.py > Graph > add_vertex

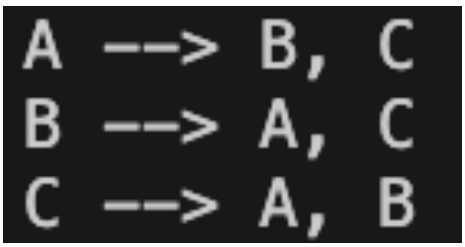
```python
# generate the code to impliment a graph using an adjacency list.
# Graph with methods to add vertices, add edges, and display connections.
class Graph:
    def __init__(self):
        self.adjacency_list = {}

    def add_vertex(self, vertex):
        if vertex not in self.adjacency_list:
            self.adjacency_list[vertex] = []

    def add_edge(self, vertex1, vertex2):
        if vertex1 in self.adjacency_list and vertex2 in self.adjacency_list:
            self.adjacency_list[vertex1].append(vertex2)
            self.adjacency_list[vertex2].append(vertex1)

    def display_connections(self):
        for vertex, edges in self.adjacency_list.items():
            print(f"{vertex} --> {', '.join(edges)}")
# Example usage:
graph = Graph()
graph.add_vertex("A")
graph.add_vertex("B")
graph.add_vertex("C")
graph.add_edge("A", "B")
graph.add_edge("A", "C")
graph.add_edge("B", "C")
graph.display_connections()
```

OutPut:



Task - 7 : Priority Queue

Prompt :

generate a code to implement a priority queue using Python's heapq module.
Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Code:

Ai-11.1.py > PriorityQueue > is_empty

```python
# generate a code to implement a priority queue using Python's heapq module.
# Implementation with enqueue (priority), dequeue (highest priority), and display methods.
import heapq
class PriorityQueue:
    def __init__(self):
        self.elements = []

    def enqueue(self, item, priority):
        heapq.heappush(self.elements, (priority, item))

    def dequeue(self):
        if not self.is_empty():
            return heapq.heappop(self.elements)[1]
        else:
            raise IndexError("Dequeue from an empty priority queue")

    def is_empty(self):
        return len(self.elements) == 0

    def display(self):
        print("Priority Queue:")
        for priority, item in sorted(self.elements):
            print(f"Priority: {priority}, Item: {item}")
# Example usage
if __name__ == "__main__":
    pq = PriorityQueue()
    pq.enqueue("Task 1", priority=2)
    pq.enqueue("Task 2", priority=1)
    pq.enqueue("Task 3", priority=3)

    pq.display()

    print("\nDequeue:", pq.dequeue())
    print("\nAfter Dequeue:")
    pq.display()
```

OutPut :

```
Priority Queue:
Priority: 1, Item: Task 2
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3

Dequeue: Task 2

After Dequeue:
Priority Queue:
Priority: 2, Item: Task 1
Priority: 3, Item: Task 3
```

Task - 8 : Deque

Prompt :

generate a code to implement a double-ended queue using collections.deque.
Insert and remove from both ends with docstrings.

Code :

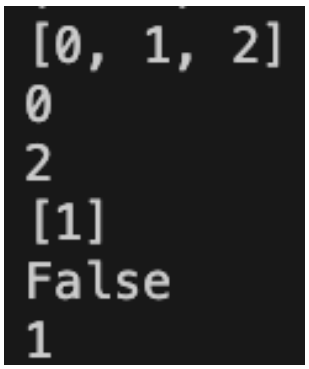
Ai-11.1.py > DoubleEndedQueue > remove_front

```python
# generate a code to implement a double-ended queue using collections.deque.
# Insert and remove from both ends with docstrings.
from collections import deque
class DoubleEndedQueue:
    """
    A double-ended queue (deque) implementation using collections.deque.
    This class allows insertion and removal of elements from both ends of the queue.
    """
    def __init__(self):
        """Initialize an empty double-ended queue."""
        self.deque = deque()

    def insert_front(self, item):
        """Insert an item at the front of the deque."""
        self.deque.appendleft(item)

    def insert_rear(self, item):
        """Insert an item at the rear of the deque."""
        self.deque.append(item)
    def remove_front(self):
        """Remove and return an item from the front of the deque. Raises IndexError if the deque is empty."""
        if not self.deque:
            raise IndexError("Deque is empty")
        return self.deque.popleft()
    def remove_rear(self):
        """Remove and return an item from the rear of the deque. Raises IndexError if the deque is empty."""
        if not self.deque:
            raise IndexError("Deque is empty")
        return self.deque.pop()
    def is_empty(self):
        """Check if the deque is empty."""
        return len(self.deque) == 0
    def size(self):
        """Return the number of items in the deque."""
        return len(self.deque)
    def __str__(self):
        """Return a string representation of the deque."""
        return str(list(self.deque))
# Example usage:
if __name__ == "__main__":
    deque = DoubleEndedQueue()
    deque.insert_rear(1)
    deque.insert_rear(2)
    deque.insert_front(0)
    print(deque)  # Output: [0, 1, 2]
    print(deque.remove_front())  # Output: 0
    print(deque.remove_rear())   # Output: 2
    print(deque)  # Output: [1]
    print(deque.is_empty())  # Output: False
    print(deque.size())  # Output: 1
```

OutPut :



Task - 9 : Real-Time Application Challenge – Choose the Right Data Structure

- **Data Structure Selection**

Feature	Chosen Data Structure	Justification
Student Attendance Tracking	**Hash Table**	Attendance requires fast lookup by student ID and date. A Hash Table provides O(1) average-time insertion and search, making it efficient for daily logs and quick record retrieval.
Event Registration System	**Binary Search Tree (BST)**	A BST allows efficient searching, insertion, and deletion (O(log n) average case). This is useful when managing participants with quick lookup and removal.
Library Book Borrowing	**Priority Queue**	Books can be prioritized by due date. A Priority Queue ensures the book with the nearest due date is accessed first, which helps in tracking overdue books efficiently.
Bus Scheduling System	**Graph**	Bus routes and stops naturally form nodes and connections. A Graph structure is ideal for representing routes and finding shortest paths between stops.
Cafeteria Order Queue	**Queue**	Orders must be served in First-In-First-Out (FIFO) order. A Queue perfectly models this real-world behavior.

Prompt :

"""

Create a Cafeteria Order Management System using a Queue data structure.

Requirements:

- Use collections.deque to implement the queue.
- Create a class called CafeteriaQueue.
- Methods:
  add_order(student_name, order_item)
  serve_order()
  display_orders()
- Follow FIFO principle.
- Include docstrings and comments.
- Add a simple menu-driven interface for testing.

"""

Code:

```python
from collections import deque
class CafeteriaQueue:
    def __init__(self):
        """Initialize an empty queue for orders."""
        self.orders = deque()
    def add_order(self, student_name, order_item):
        """Add a new order to the queue."""
        self.orders.append((student_name, order_item))
        print(f"Order added: {student_name} - {order_item}")
    def serve_order(self):
        """Serve the next order in the queue."""
        if self.orders:
            student_name, order_item = self.orders.popleft()
            print(f"Order served: {student_name} - {order_item}")
        else:
            print("No orders to serve.")
    def display_orders(self):
        """Display all current orders in the queue."""
        if self.orders:
            print("Current Orders:")
            for student_name, order_item in self.orders:
                print(f"{student_name} - {order_item}")
        else:
            print("No current orders.")
def main():
    cafeteria_queue = CafeteriaQueue()
    while True:
        print("\nMenu:")
        print("1. Add Order")
        print("2. Serve Order")
        print("3. Display Orders")
        print("4. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            student_name = input("Enter student name: ")
            order_item = input("Enter order item: ")
            cafeteria_queue.add_order(student_name, order_item)
        elif choice == '2':
            cafeteria_queue.serve_order()
        elif choice == '3':
            cafeteria_queue.display_orders()
        elif choice == '4':
            print("Exiting the system.")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

OutPut :

```
Menu:
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 1
Enter student name: harini
Enter order item: shoes
Order added: harini - shoes

Menu:
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 2
Order served: harini - shoes

Menu:
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 3
No current orders.

Menu:
1. Add Order
2. Serve Order
3. Display Orders
4. Exit
Enter your choice: 4
Exiting the system.
```

Selected Feature for Implementation :

I recommend implementing **Cafeteria Order Queue (Queue)** because:

- It’s simple
- Clearly demonstrates FIFO
- Easy for Copilot to generate correctly
- Easy to explain during evaluation

Task - 10 : Smart E-Commerce Platform – Data Structure Challenge

Data Structure Selection Table :

Feature	Chosen Data Structure	Justification
Shopping Cart Management	**Linked List**	A shopping cart requires dynamic addition and removal of products. A Linked List allows efficient insertion and deletion without shifting elements, making it suitable for frequently changing cart items.
Order Processing System	**Queue**	Orders must be processed in First-In-First-Out (FIFO) order. A Queue ensures fairness and maintains the exact sequence in which customers place their orders.
Top-Selling Products Tracker	**Priority Queue**	Products need to be ranked by sales count. A Priority Queue efficiently keeps the highest-selling products at the top, allowing quick retrieval of top performers.
Product Search Engine	**Hash Table**	Product lookup by product ID must be very fast. A Hash Table provides O(1) average time complexity for search, insertion, and deletion, making it ideal for quick product retrieval.
Delivery Route Planning	**Graph**	Warehouses and delivery locations form nodes connected by routes (edges). A Graph structure is perfect for representing networks and computing shortest delivery paths.

Prompt :

""" Create an E-Commerce Order Processing System using a Queue.
Requirements:
- Use collections.deque.
- Create a class called OrderProcessingSystem.
- Each order should include:
order_id (auto increment) , customer_name , product_name , price , timestamp
- Methods:
place_order() , process_order() , cancel_order(order_id) , display_pending_orders()
- Follow FIFO principle.
- Add proper docstrings and comments.
- Include a menu-driven interface. """

Code:

Ai-11.1.py > ...

```python
from collections import deque
import time
class Order:
    """Class representing an individual order."""
    def __init__(self, order_id, customer_name, product_name, price):
        self.order_id = order_id
        self.customer_name = customer_name
        self.product_name = product_name
        self.price = price
        self.timestamp = time.time()
    def __str__(self):
        return (f"Order ID: {self.order_id}, Customer: {self.customer_name}, "
                f"Product: {self.product_name}, Price: ${self.price:.2f}, "
                f"Timestamp: {time.ctime(self.timestamp)}")
class OrderProcessingSystem:
    """Class representing the order processing system using a queue."""
    def __init__(self):
        self.orders = deque()
        self.order_id_counter = 1
    def place_order(self, customer_name, product_name, price):
        """Place a new order in the system."""
        order = Order(self.order_id_counter, customer_name, product_name, price)
        self.orders.append(order)
        print(f"Order placed: {order}")
        self.order_id_counter += 1
    def process_order(self):
        """Process the next order in the queue."""
        if self.orders:
            order = self.orders.popleft()
            print(f"Processing order: {order}")
        else:
            print("No orders to process.")
    def cancel_order(self, order_id):
        """Cancel an order by its ID."""
        for order in list(self.orders):
            if order.order_id == order_id:
                self.orders.remove(order)
                print(f"Order cancelled: {order}")
                return
        print(f"No order found with ID: {order_id}")
    def display_pending_orders(self):
        """Display all pending orders in the queue."""
        if self.orders:
            print("Pending Orders:")
            for order in self.orders:
                print(order)
        else:
            print("No pending orders.")
def main():
    """Main function to run the menu-driven interface."""
    system = OrderProcessingSystem()
    while True:
        print("\nMenu:")
        print("1. Place Order")
        print("2. Process Order")
        print("3. Cancel Order")
        print("4. Display Pending Orders")
        print("5. Exit")
        choice = input("Enter your choice: ")
        if choice == '1':
            customer_name = input("Enter customer name: ")
            product_name = input("Enter product name: ")
            price = float(input("Enter price: "))
            system.place_order(customer_name, product_name, price)
        elif choice == '2':
            system.process_order()
        elif choice == '3':
            order_id = int(input("Enter order ID to cancel: "))
            system.cancel_order(order_id)
        elif choice == '4':
            system.display_pending_orders()
        elif choice == '5':
            print("Exiting the system.")
            break
        else:
            print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

OutPut :

```
Menu:
1. Place Order
2. Process Order
3. Cancel Order
4. Display Pending Orders
5. Exit
Enter your choice: 1
Enter customer name: vishnu
Enter product name: watch
Enter price: 3590
Order placed: Order ID: 1, Customer: vishnu, Product: watch, Price: $3590.00, Timestamp: Mon Feb 23 18:46:38 2026

Menu:
1. Place Order
2. Process Order
3. Cancel Order
4. Display Pending Orders
5. Exit
Enter your choice: 2
Processing order: Order ID: 1, Customer: vishnu, Product: watch, Price: $3590.00, Timestamp: Mon Feb 23 18:46:38 2026

Menu:
1. Place Order
2. Process Order
3. Cancel Order
4. Display Pending Orders
5. Exit
Enter your choice: 3
Enter order ID to cancel: 4
No order found with ID: 4

Menu:
1. Place Order
2. Process Order
3. Cancel Order
4. Display Pending Orders
5. Exit
Enter your choice: 5
Exiting the system.
```

Selected Feature for Implementation

I recommend implementing **Order Processing System (Queue)** because:

- Clear real-world FIFO logic
- Easy to demonstrate
- Clean implementation
- Perfect for Copilot generation